

Юнит-тестирование и системы сборки

Юрий Литвинов
y.litvinov@spbu.ru

03.10.2026

Системы сборки

- ▶ Нужны, чтобы сборка не зависела от среды разработки
 - ▶ Воспроизводимость сборки
 - ▶ «Пакетный режим», CI
- ▶ Умеют вычислять зависимости между компонентами, управлять внешними зависимостями
 - ▶ И автоматически скачивать нужные пакеты при сборке
- ▶ Не только, собственно, сборка: тестирование, создание документации, инсталляторов, отчётов и т.д.
- ▶ Maven
- ▶ Gradle

Maven, основные принципы

- ▶ Декларативность
- ▶ Convention Over Configuration
 - ▶ Стандартная структура папок
- ▶ Плагины
- ▶ Project Object Model
- ▶ Жизненный цикл
- ▶ Координаты
 - ▶ groupId
 - ▶ artifactId
 - ▶ version
- ▶ Репозиторий и on-demand-скачивание пакетов
 - ▶ СИстема поиска пакетов — <https://mvnrepository.com/>
 - ▶ Локальный кеш: `/.m2/repository`

Maven, стандартная структура папок

- ▶ pom.xml
- ▶ src
 - ▶ main
 - ▶ java
 - ▶ resources
 - ▶ test
 - ▶ java
 - ▶ resources
- ▶ target

Gradle

- ▶ Более легковесный и современный, чем Maven
- ▶ Декларативный предметно-ориентированный язык на основе Groovy или Kotlin
- ▶ Использует Convention Over Configuration, но легко переопределить
 - ▶ Та же структура папок по умолчанию
- ▶ Использует репозиторий Maven для динамической загрузки зависимостей
- ▶ build.gradle, settings.gradle
- ▶ gradle-wrapper
 - ▶ gradlew, gradlew.bat
 - ▶ gradle-wrapper.jar, gradle-wrapper.properties

Модульное тестирование, информация к размышлению

- ▶ Программа из сотни строк может иметь 10^{18} путей исполнения
 - ▶ Полный перебор занял бы слишком много времени, а в реальных программах путей исполнения ещё больше
- ▶ После передачи на тестирование в программах в среднем от 1 до 3 ошибок на 100 строк кода
- ▶ В процессе разработки — 1.5 ошибок на 1 строку кода (!)
- ▶ Если для исправления ошибки надо изменить не более 10 операторов, с первого раза это делают правильно в 50% случаев
- ▶ Если для исправления ошибки надо изменить не более 50 операторов, с первого раза это делают правильно в 20% случаев

JUnit 6

- ▶ JUnit — самая известная библиотека юнит-тестирования для Java
- ▶ JUnit 6 — его последняя версия (часто до сих пор используется JUnit 5 или даже 4)
- ▶ Отлично интегрируется с IDE
 - ▶ Генерация заголовков тестов, запуск тестов прямо из редактора и т.д.
 - ▶ Справедливости ради, тесты хорошо пишутся LLMками, так что это не так важно
- ▶ Тест — это отдельный класс, в котором есть методы, помеченные аннотацией `@Test`
- ▶ Внутри теста обычно три фазы
 - ▶ Настройка тестового окружения
 - ▶ Выполнение действия, которое хотим тестировать
 - ▶ Проверка результатов
 - ▶ Вызовы `assertEquals`, `assertNull`, `assertTrue` и т.д.

Ещё аннотации

- ▶ **@BeforeEach** — метод, запускающийся перед каждым тестом
- ▶ **@AfterEach** — метод, запускающийся после каждого теста
- ▶ **@BeforeAll** — метод, запускающийся перед всеми тестами в классе (должен быть static)
- ▶ **@AfterAll** — метод, запускающийся после всех тестов в классе (должен быть static)
- ▶ **@Disabled** — тест отключён и не запускается

Assertions

Наиболее интересные

- ▶ `assertEquals`
 - ▶ Обратите внимание на порядок параметров, сначала ожидаемое, затем то, что получилось
- ▶ `assertArrayEquals`
- ▶ `assertTrue/assertFalse`
- ▶ `assertNull/assertNotNull`
- ▶ `assertThrows`
- ▶ `fail`

Best practices

- ▶ Независимость тестов
 - ▶ Желательно, чтобы поломка одного куска функциональности ломала один тест
- ▶ Тесты должны работать быстро
 - ▶ И запускаться после каждой сборки
 - ▶ Continuous Integration
- ▶ Тестов должно быть много
 - ▶ Следите за Code coverage, это хороший индикатор непокрытого и мёртвого кода
- ▶ Каждый тест должен проверять конкретный тестовый сценарий
 - ▶ Нельзя ловить исключения в тесте без крайней нужды
 - ▶ С большой осторожностью пользоваться Random-ом
- ▶ Именование тестов: <http://stackoverflow.com/questions/155436/unit-test-naming-best-practices>
- ▶ Test-driven development

Hamcrest

- ▶ Библиотека матчеров — объектов, представляющих логическое условие
- ▶ Нужна прежде всего для удобства записи assert-ов в модульных тестах
 - ▶ И для читаемости, что немаловажно
- ▶ В JUnit 4 входила в зависимости фреймворка, дальше — надо добавлять отдельно

Hamcrest, пример

- ▶ `assertThat(someString, is(not(equalTo(someOtherString))));`
- ▶ `assertThat(list, everyItem(greaterThan(1)));`
- ▶ `assertThat(cat.getKittens(), hasItem(someKitten));`
- ▶ `assertThat("test", anyOf(is("testing"), containsString("est")));`
- ▶ `assertThat(x,
 allOf(greaterThan(0), lessThanOrEqualTo(10)));`

AssertJ

- ▶ Библиотека fluent assertions — ещё более человекочитаемый синтаксис, чем Hamcrest

- ▶ Пример (из документации):

```
assertThat(frodo.getName()).isEqualTo("Frodo");  
assertThat(frodo).isNotEqualTo(sauron);
```

```
assertThat(frodo.getName()).startsWith("Fro")  
    .endsWith("do")  
    .isEqualToIgnoringCase("frodo");
```

```
assertThat(fellowshipOfTheRing)  
    .hasSize(9)  
    .contains(frodo, sam)  
    .doesNotContain(sauron);
```

```
assertThatThrownBy(() ->  
    { throw new Exception("boom!"); }).hasMessage("boom!");
```

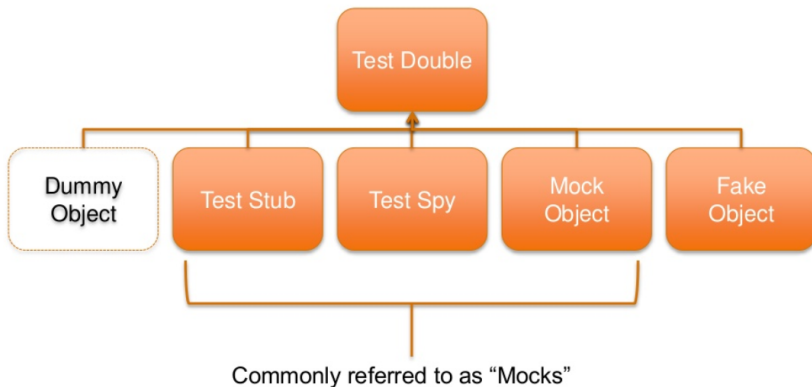
Truth

- ▶ Библиотека fluent assertions от Google, альтернатива AssertJ
 - ▶ Схожая философия, Truth более простой (что может быть как плюсом, так и минусом)
- ▶ Пример (из документации):
`assertThat(projectsByTeam())
 .valuesForKey("corelibs")
 .containsExactly("guava", "dagger", "truth", "auto", "caliper");`

Mock-объекты

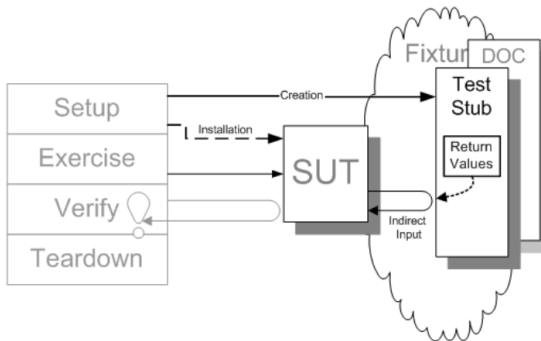
- ▶ Объекты-заглушки, симулирующие поведение реальных объектов и контролирующие обращения к своим методам
 - ▶ Как правило, такие объекты создаются с помощью библиотек
- ▶ Используются, когда реальные объекты использовать
 - ▶ Слишком долго
 - ▶ Слишком опасно
 - ▶ Слишком трудно
 - ▶ Для добавления детерминизма в тестовый сценарий
 - ▶ Пока реального объекта ещё нет
 - ▶ Для изоляции тестируемого объекта
- ▶ Для использования mock-объекта требуется какой-то механизм внедрения объекта
 - ▶ Как правило, как параметр конструктора или через поле

Объекты-заглушки



Stubs

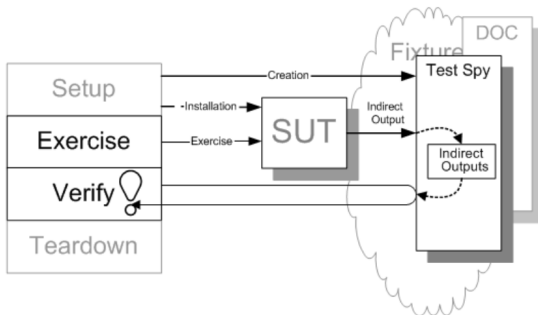
- ▶ Захардкоженные объекты, реализующие нужный интерфейс
- ▶ Наивная или вообще отсутствующая реализация
 - ▶ Возможно какие-то assert'ы или полезная логика для теста



© <http://xunitpatterns.com>

Spies

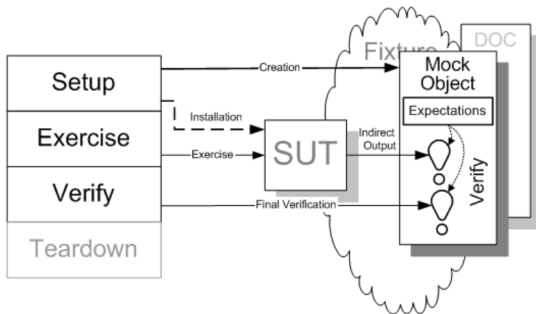
- ▶ Запуск теста, затем проверка условий и ограничений



© <http://xunitpatterns.com>

Mocks

► Конфигурирование объекта перед запуском теста



© <http://xunitpatterns.com>

Mockito

- ▶ Один из самых известных фреймворков для мок-объектов
 - ▶ Но далеко не единственный
- ▶ <http://site.mockito.org/>
- ▶ <https://github.com/mockito/mockito>

Классический тест

@Test

```
public void test() throws Exception {  
    // Arrange  
    var testee = new UnitToTest();  
    var helper = new Helper();  
    // Act  
    testee.doSomething(helper);  
    // Assert  
    assertTrue(helper.somethingHappened());  
}
```

Тест с mock-объектом

@Test

```
public void test() throws Exception {  
    // Arrange, prepare behaviour  
    Helper aMock = mock(Helper.class);  
    when(aMock.isCalled()).thenReturn(true);  
    // Act  
    testee.doSomething(aMock);  
    // Assert - verify interactions (optional)  
    verify(aMock).isCalled();  
}
```

Создание mock-объектов

- ▶ `IMyClass myMock = mock(IMyClass.class)`
- ▶ `MyClass myMock = mock(MyClass.class)`
- ▶ `@Mock private MyClass myMock;`
 - ▶ `@Before public void setup() {`
 `MockitoAnnotations.openMocks(this);`
 `}`

when(...)

- ▶ `when(mock.someMethod()).thenReturn(10);`
- ▶ `when(mock.someMethod("some arg"))
 .thenThrow(new RuntimeException());`
- ▶ `when(mock.someMethod(anyString())).thenReturn(42);`
- ▶ `when(mock.someMethod(anyInt())).thenReturn("element");`
- ▶ `when(mock.someMethod(argThat(list -> list.size() < 3)))
 .thenReturn(null);`

Последовательные вызовы

- ▶ `when(mock.someMethod("some arg"))
 .thenThrow(new RuntimeException())
 .thenReturn("foo");`
- ▶ `when(mock.someMethod("some arg"))
 .thenReturn("one", "two", "three");`
- ▶ Но:
`when(mock.someMethod("some arg")).thenReturn("one")
when(mock.someMethod("some arg")).thenReturn("two")`

Подмена действия

```
when(mock.someMethod(anyString())).thenAnswer(new Answer() {  
    Object answer(InvocationOnMock invocation) {  
        Object[] args = invocation.getArguments();  
        return "called with arguments: " + Arrays.toString(args);  
    }  
});
```

//the following prints "called with arguments: [foo]"

```
System.out.println(mock.someMethod("foo"));
```

verify(...)

- ▶ `verify(mock).someMethod();`
- ▶ `verify(mock, timeout(100)).someMethod();`
- ▶ `verify(mock).someMethod(
 anyInt(), anyString(), eq("third argument"));`
- ▶ `verify(mock).someMethod(
 argThat(someString -> someString.length() > 5));`

Повторные вызовы

- ▶ `verify(mock, times(2)).someMethod();`
- ▶ `verify(mock, never()).someMethod();`
 - ▶ `verifyNoInteractions(mockTwo);`
- ▶ `verify(mock, atLeastOnce()).someMethod();`
- ▶ `verify(mock, atLeast(2)).someMethod();`
- ▶ `verify(mock, atMost(5)).someMethod();`
- ▶ `InOrder inOrder = inOrder(mock);`
`inOrder.verify(mock).someMethod("was called first");`
`inOrder.verify(mock).someMethod("was called second");`

spy(...)

```
List list = new LinkedList();
```

```
List spy = spy(list);
```

```
when(spy.size()).thenReturn(100);
```

```
spy.add("one");
```

```
spy.add("two");
```

```
//prints "one" - the first element of a list
```

```
System.out.println(spy.get(0));
```

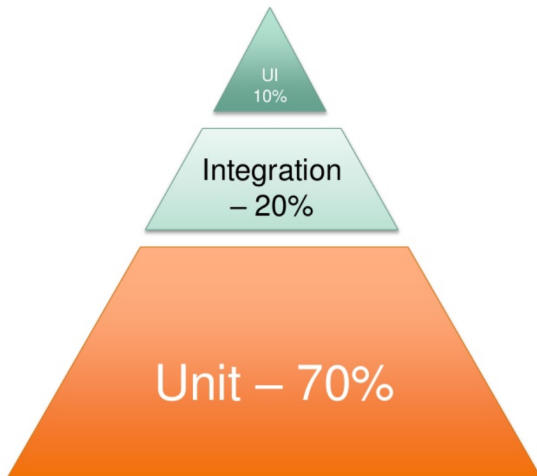
```
//size() method was stubbed - 100 is printed
```

```
System.out.println(spy.size());
```

```
verify(spy).add("one");
```

```
verify(spy).add("two");
```

Примерное соотношение объёма тестов в проекте



Задание на остаток пары

- ▶ Поделиться на команды по 2-3 человека
- ▶ Написать модульные тесты на `StringSet` и `List` с предыдущей пары
- ▶ Используйте JUnit, одну из библиотек матчеров на выбор и Mockito
 - ▶ Mockito для изоляции тестов на хеш-таблицу, чтобы они не зависели от списка
- ▶ Используйте правильное именование тестов и подход «Arrange-Act-Assert»